

Tarea 2

DISEÑO Y VERIFICACIÓN DE UN DIVISOR SECUENCIAL
BASADO EN SUMAS Y DESPLAZAMIENTOS

DANIEL MAYORAL BAÑOS

Objetivos

- Implementación directa de ASM mediante systemVerilog
- Ejercitar la realización de bancos de pruebas con systemVerilog con RCSG, cobertura funcional, clases, modelos de referencia, interfaces, bloques de reloj y aserciones.

Índice

1. Diseño del Divisor Algorítmico
2. Banco de Pruebas de nivel 1 (tb_DivSec_lvl1)
3. Banco de Pruebas de nivel 2 (tb_DivSec_lvl2)
4. Banco de Pruebas de nivel 3 (tb_DivSec_lvl3)
5. Diseño del Divisor Segmentado

Diseño del Divisor Algorítmico

El módulo Divisor_Algoritmico implementa un divisor basado en un algoritmo secuencial que calcula el cociente (Coc) y el residuo (Res) de la división entre dos números (Num y Den). El diseño está parametrizado para manejar números de tamaño configurable (tamanyo, por defecto 32 bits). El cálculo se realiza en varios ciclos de reloj, utilizando un esquema de control por estados.

Entradas y Salidas:

- Entradas:
 - CLK: Señal de reloj para sincronización.
 - RSTa: Reset asíncrono, que inicializa el módulo.
 - Start: Señal para iniciar la operación de división.
 - Num: Numerador de la división (dato de entrada).
 - Den: Denominador de la división (dato de entrada).
- Salidas:
 - Coc: Cociente resultante de la división.
 - Res: Residuo de la división.
 - Done: Indica cuando la operación de división ha finalizado.

Parámetro:

- tamanyo: Define el ancho de los datos (Num, Den, Coc, Res). Permite ajustar la precisión y tamaño de las operaciones.

Variables Internas

- SignNum y SignDen: Flags que almacenan el signo del numerador y denominador respectivamente, para manejar números con signo.
- ACCU: Acumulador interno que almacena los restos parciales durante la operación de división.
- Q: Registro que contiene el cociente parcial.
- M: Magnitud del denominador (transformada a valor absoluto si es necesario).
- Count: Contador decreciente que controla el número de iteraciones necesarias para completar la operación de división (inicializado a tamanyo-1).
- state: Máquina de estados para controlar las etapas del algoritmo. Los estados son:
 - d0: Inicialización.
 - d1: Desplazamiento y preparación de datos.
 - d2: Comparación, actualización de acumulador y decremento del contador.
 - d3: Ajuste final y generación del resultado.

Funcionamiento:

Estado d0 (Inicialización):

Espera a que Start se active. Si se activa, inicializa los registros internos:

- Convierte Num y Den a valores absolutos si tienen signo negativo.
- Carga ACCU y Q con valores iniciales.
- Establece Count en tamaño-1.
- Determina el signo del resultado final usando SignNum y SignDen.
- Cambia al estado d1 para comenzar el cálculo.

Estado d1 (Desplazamiento):

Realiza un desplazamiento lógico de ACCU y Q para preparar el siguiente paso del cálculo. Cambia al estado d2.

Estado d2 (Comparación y Resta):

Compara ACCU con M; si $ACCU \geq M$, resta M de ACCU y actualiza el cociente parcial (Q). Decrementa el contador (Count) y, si Count llega a 0, pasa al estado d3. De lo contrario, regresa al estado d1 para continuar el cálculo.

Estado d3 (Finalización):

Ajusta el signo del cociente (Coc) y el residuo (Res) según los valores de SignNum y SignDen. Establece Done en alto para indicar que el cálculo ha finalizado y regresa al estado d0 para esperar una nueva operación.

Prestaciones del diseño:

Las prestaciones alcanzadas para este diseño son:

Frecuencia máxima alcanzada (Slow 1200mV 85°): 160.26 MHz

Banco de Pruebas de nivel 1 (tb_DivSec_lvl1)

El módulo tb_DivSec_lvl1 es un testbench simple que verifica el correcto funcionamiento del módulo Divisor_Algoritmico. Se utiliza para simular diferentes casos de división, tanto con números positivos como negativos, y se comprueban los resultados obtenidos mediante impresiones en consola.

- **Objetivo:** Probar el módulo Divisor_Algoritmico con entradas de prueba sencillas y validar que los resultados de cociente (Coc) y residuo (Res) son correctos en distintas situaciones.
- **Nivel de Verificación:** Básico (nivel 1), limitado a casos extrictos.

Componentes del Testbench:

1. Parámetros

- **tamanyo:** Parámetro que define el tamaño de los datos (32 bits en este caso). Este parámetro es el mismo que el utilizado en el módulo Divisor_Algoritmico.

2. Señales de Entrada y Salida

- **Entradas de prueba:**
 - CLK: Señal de reloj que controla el flujo secuencial.
 - RSTa: Reset asíncrono para inicializar el módulo bajo prueba.
 - Start: Señal que indica el inicio de la operación.
 - Num y Den: Datos de entrada que representan el numerador y denominador respectivamente.
- **Salidas observadas:**
 - Coc y Res: Resultados del cociente y residuo calculados por el módulo bajo prueba.
 - Done: Señal que indica la finalización del cálculo.

3. Instanciación del Módulo Bajo Prueba (DUT)

El módulo Divisor_Algoritmico es instanciado con las señales de prueba conectadas a sus respectivas entradas y salidas.

4. Generación de la Señal de Reloj

El reloj se genera con un período de 10 unidades de tiempo mediante un bloque always:

```
always #5 CLK = ~CLK;
```

5. Procedimiento de Inicialización y Simulación

- Reset: El reset (RSTa) es activado brevemente al inicio para garantizar que el DUT comience en un estado conocido.
- Casos de prueba: Se definen varios escenarios de prueba, cada uno configurando los valores de Num, Den, y activando la señal Start para iniciar la operación. Cada caso incluye:
 - Espera a que el módulo termine (señal Done).
 - Impresión de los resultados usando \$display.
- Finalización: Tras completar todas las pruebas, la simulación termina con \$finish.

Casos de Prueba:

1. División 100 / 2:
 - Entrada: Num = 100, Den = 2.
 - Resultado esperado: Coc = 50, Res = 0.
2. División -100 / 2:
 - Entrada: Num = -100, Den = 2.
 - Resultado esperado: Coc = -50, Res = 0.
3. División 45 / -5:
 - Entrada: Num = 45, Den = -5.
 - Resultado esperado: Coc = -9, Res = 0.
4. División -45 / -5:
 - Entrada: Num = -45, Den = -5.
 - Resultado esperado: Coc = 9, Res = 0.
5. División con residuo (37 / 4):
 - Entrada: Num = 37, Den = 4.
 - Resultado esperado: Coc = 9, Res = 1.

Banco de Pruebas de nivel 2 (tb_DivSec_lvl2)

El módulo tb_DivSec_lvl2 es un testbench medio diseñado para verificar el correcto funcionamiento del módulo Divisor_Algoritmico. Emplea generación de casos de prueba aleatorios (RSCG) y cobertura funcional mediante covergroups. Esto permite probar escenarios más variados y garantizar una validación exhaustiva del diseño.

- **Objetivo:** Realizar una verificación de nivel 2 sobre el módulo Divisor_Algoritmico, incluyendo pruebas con valores aleatorios controlados por restricciones y cobertura funcional.
- **Características:**
 - Uso de generación aleatoria con restricciones (rand y constraint).
 - Cobertura funcional mediante coverpoints y cross coverage.
 - Validación automatizada de los resultados esperados frente a los resultados calculados por el DUT.
- **Nivel de Verificación:** Avanzado (Nivel 2). Permite explorar automáticamente un espacio más amplio de posibles entradas y garantiza que se ejerciten escenarios límite y combinaciones críticas.

Componentes del Testbench

1. Parámetros

- **tamanyo:** Define el ancho de las señales en bits (32 bits por defecto). Este valor coincide con el usado en el DUT.

2. Señales de Entrada y Salida

- **Entradas de prueba:**
 - CLK: Señal de reloj que sincroniza el DUT.
 - RSTa: Reset asíncrono para inicializar el módulo bajo prueba.
 - Start: Señal que da inicio a la operación de división.
 - Num y Den: Entradas que representan el numerador y denominador, respectivamente.
- **Salidas observadas:**
 - Coc: Cociente calculado por el módulo.
 - Res: Residuo calculado por el módulo.
 - Done: Señal que indica la finalización de la operación.

3. Generación Aleatoria con Restricciones (RSCG)

Se utiliza una clase llamada Bus que genera valores aleatorios para las señales Num y Den. Las restricciones (constraints) garantizan que:

- Relación de signo: Los valores aleatorios de Den y Num pueden ser positivos o negativos, pero siempre cumplen las restricciones definidas en SignRelationship.
- No se permite denominador cero: Restringido por constraint NoZero, ya que dividir entre 0 no es válido.

4. Cobertura Funcional

El covergroup NumDen_CG evalúa las combinaciones de valores de entrada (Num y Den), con:

- Bins: Define intervalos de interés como:
 - Valores positivos, negativos, máximos, mínimos y cero.
- Cross Coverage: Evalúa combinaciones cruzadas entre Num y Den para garantizar que se ejercitan múltiples combinaciones de valores.

5. Validación de Resultados

Se comparan los valores calculados (Coc, Res) con los valores esperados, generados en el testbench mediante operaciones aritméticas en SystemVerilog. La validación incluye:

- Cálculo del cociente y residuo esperados.
- Comparación directa con las salidas del DUT.
- Reporte de errores si hay discrepancias.

Flujo de la Simulación

1. Inicialización:
 - Se configuran las señales iniciales y se activa/desactiva el reset para preparar el DUT.
 - Se crea una instancia de la clase Bus para generar valores aleatorios.
2. Generación de Casos Aleatorios:
 - Se generan 50 pruebas aleatorias utilizando la clase Bus.
 - Los valores aleatorios son asignados a Num y Den del DUT.
 - Se activa la señal Start para iniciar cada operación.
3. Cobertura:
 - Se muestrean los valores generados en el covergroup después de asignarlos al DUT.
4. Validación:
 - Se espera a que la señal Done indique la finalización de la operación.
 - Los resultados del DUT (Coc y Res) se comparan con los valores esperados.
5. Reporte:
 - Se genera un reporte de éxito o error para cada caso de prueba.
 - Al final de la simulación, se muestra el porcentaje de cobertura funcional alcanzado.

6. Finalización:

- Se finaliza la simulación con \$finish.

Mensajes de Salida:

- Resultados de las Pruebas: Cada caso imprime si la prueba fue exitosa o fallida, mostrando los valores de entrada y salida.
- Cobertura Funcional: Se reporta el porcentaje de cobertura alcanzado

Banco de Pruebas de nivel 3 (tb_DivSec_lvl3)

El testbench tb_DivSec_lvl3 es un entorno de verificación de nivel 3 diseñado para el módulo Divisor_Algoritmico. Utiliza conceptos avanzados de verificación en SystemVerilog como:

- Interfaces y Modports: Simplifican la conexión entre el testbench y el módulo bajo prueba (DUT).
- Clocking Blocks: Aseguran sincronización precisa en las lecturas y escrituras.
- Scoreboard: Valida automáticamente las salidas del DUT comparándolas con resultados esperados.
- Random Stimulus with Constrained Generation (RSCG): Genera estímulos aleatorios bajo restricciones definidas.
- Cobertura Funcional (covergroups): Garantiza que todas las combinaciones de interés se ejercitan durante la simulación.

Estructura del Testbench

1 Parámetros

- **tamanyo**: Define el ancho de los datos (32 bits por defecto).

2 Interfaz (DivSecIF)

La interfaz centraliza las señales del DUT y proporciona dos clocking blocks:

- md (monitor data): Sincroniza las entradas y salidas del DUT.
- sd (stimulus driver): Facilita la generación y aplicación de estímulos.

La interfaz incluye modports para definir roles específicos:

- Monitor: Para el acceso de lectura al DUT.
- test: Para la escritura de estímulos.
- duv: Para la conexión directa al DUT.

3 DUT Wrapper (duv)

El wrapper conecta las señales del DUT con la interfaz

4 Clase Bus

Genera estímulos aleatorios con restricciones para las entradas Num y Den:

- Restricciones (constraint): Aseguran que las combinaciones sean válidas:
 - Evita divisiones por cero (NoZero).
 - Controla las combinaciones de signos entre Num y Den.

5 Scoreboard

Valida las salidas del DUT comparándolas con los resultados ideales calculados internamente:

- Utiliza colas para almacenar los valores esperados de cociente y residuo.
- Realiza una comparación directa entre los resultados del DUT y los valores calculados

6 Cobertura Funcional

Se define un covergroup para garantizar que las pruebas exploran:

- Diferentes valores de Num y Den (positivos, negativos, máximos, mínimos, y ceros ignorados).
- Combinaciones cruzadas (cross coverage) entre las entradas.

7 Simulación

Inicialización

- Se configuran las señales iniciales (RSTa, CLK).
- Se crean instancias del Scoreboard, generador de estímulos (Bus) y el covergroup.

Aplicación de Estímulos

El programa de estímulos ejecuta las pruebas en seis fases:

1. Primera Parte ($\text{Den} > 0$, $\text{Num} > 0$): Explora combinaciones con ambas entradas positivas.
2. Segunda Parte ($\text{Den} < 0$, $\text{Num} < 0$): Prueba ambas entradas negativas.
3. Tercera Parte ($\text{Den} > 0$, $\text{Num} < 0$): Num negativo y Den positivo.
4. Cuarta Parte ($\text{Den} < 0$, $\text{Num} > 0$): Num positivo y Den negativo.
5. Quinta Parte (NoZero): Explora denominadores no nulos sin restricciones adicionales.
6. Sexta Parte (Aleatorio): Pruebas completamente aleatorias con restricciones mínimas.

En cada fase:

- Se activan o desactivan restricciones en la clase Bus.
- Se generan valores aleatorios que se aplican al DUT.
- Se verifica que las combinaciones cruzadas en el covergroup alcanzan el porcentaje esperado de cobertura.

Validación y Reporte

- El Scoreboard valida cada salida del DUT.
- Se imprime la cobertura funcional acumulada al final

Diseño del Divisor Segmentado

El módulo Divisor_Segmentado implementa una división secuencial utilizando una arquitectura segmentada que permite dividir el cálculo en múltiples etapas para mejorar el rendimiento en diseños con altas frecuencias de reloj. Esto se logra desplazando las operaciones a través de registros intermedios que sincronizan y propagan las señales del cálculo.

- **Objetivo:** Realizar la división entre dos números de 32 bits (o parametrizables) utilizando una arquitectura segmentada.
- **Características Clave:**
 - **Segmentación:** El cálculo se realiza en varias etapas, propagando los datos entre registros en cada ciclo de reloj.
 - **Manejo de Signos:** Soporta operaciones con números con signo mediante el uso del complemento a dos.
 - **Aserción para Validar el Denominador:** Garantiza que el denominador nunca sea cero durante las operaciones.
 - **Configurabilidad:** El ancho de las señales es parametrizable mediante el parámetro tamaño.

Entradas y Salidas:

- **Entradas:**
 - **CLK:** Señal de reloj que sincroniza las operaciones.
 - **RSTa:** Reset asíncrono para inicializar el módulo.
 - **Start:** Señal que inicia la operación de división.
 - **Num:** Numerador de la división (32 bits por defecto).
 - **Den:** Denominador de la división (32 bits por defecto).
- **Salidas:**
 - **Coc:** Cociente calculado por el módulo (32 bits por defecto).
 - **Res:** Residuo calculado por el módulo (32 bits por defecto).
 - **Done:** Señal que indica la finalización de la operación.

Parámetros:

- **tamaño:** Define el ancho de los datos, por defecto 32 bits. Este parámetro determina la longitud de los registros internos utilizados para realizar la operación.
- **etapas:** Define el número total de etapas en el diseño. Se calcula como $2 * \text{tamaño} + 1$, lo que asegura que cada bit del numerador se procese adecuadamente, incluyendo el bit de signo.

Estructura Interna:

1. Registros Segmentados:
 - ACCU: Acumulador parcial que almacena restos intermedios durante la división.
 - Q: Registro que contiene el cociente parcial.
 - M: Magnitud del denominador.
 - SignNum y SignDen: Registros que propagan los signos del numerador y denominador a través de las etapas.
 - Start_aux: Registro que sincroniza la señal de inicio (Start) a través de las etapas.
2. Segmentación:
 - Cada etapa realiza un paso del cálculo de división.
 - Las operaciones incluyen desplazamientos lógicos (ACCU y Q) y comparaciones para determinar si el acumulador parcial es mayor o igual al denominador.
 - Los resultados de cada etapa se almacenan en los registros segmentados, que se propagan al siguiente nivel.
3. Manejo de Signos:
 - El numerador y denominador se convierten a su magnitud absoluta si tienen signo negativo.
 - Al final del cálculo, se ajusta el signo del cociente (Coc) y del residuo (Res) según el signo del numerador y denominador originales.
4. Validación del Denominador:
 - Se utiliza una aserción para garantizar que el denominador (Den) nunca sea cero cuando la señal Start está activa:

Flujo de Operación

1. Inicialización:
 - Cuando se activa el reset (RSTa), todos los registros internos se inicializan a cero.
2. Inicio de la División:
 - La señal Start activa la operación de división, inicializando los registros en la etapa más alta (etapas-1):
 - ACCU se inicializa a cero.
 - Q se inicializa con el valor absoluto del numerador (Num).
 - M se inicializa con el valor absoluto del denominador (Den).
 - Los signos de Num y Den se almacenan en SignNum y SignDen.
3. Propagación Segmentada:
 - Cada ciclo de reloj, los registros en cada etapa se actualizan:
 - Los datos de la etapa $i+1$ se transfieren a la etapa i .

- Si la etapa actual es par, se realiza una comparación para determinar si ACCU es mayor o igual a M.
 - Si es cierto, se actualiza el acumulador restando M y se incrementa el cociente parcial.
 - Si la etapa actual es impar, se realiza un desplazamiento lógico de ACCU y Q.
4. Finalización:
- Después de completar todas las etapas, los resultados finales (Coc y Res) se encuentran en los registros de la etapa 0.
 - La señal Done se activa, indicando que la operación ha finalizado.
5. Ajuste del Signo:
- El cociente (Coc) y el residuo (Res) se ajustan a su signo correcto según los valores originales de SignNum y SignDen.

Verificación:

Para la verificación de este modulo hemos empleado el mismo banco de pruebas que el utilizado para el divisor algorítmico, dando pie a un nuevo testbench (tb_DivSeg).

Prestaciones del diseño:

Las prestaciones alcanzadas para este diseño son:

Frecuencia máxima alcanzada (Slow 1200mV 85º): 157.11 MHz